Gemini and CoPilot were both given the prompt below. Both GenAIs were accessed on a KU Linux machine using Firefox browser.

Prompt:

- “The attached C++ file contains a program that solves sudoku puzzles from txt files. Modify the program to write the solutions of each puzzle into a txt file named solutionN.txt, where N is the number of the current puzzle file. If no solutions are found, write the line “No solution found.” to the file. If solutions are found, write the solved puzzle(s) to the file, followed by a line stating “Solutions found: n”, where n is the number of solutions found. Clearly indicate modifications and additions with detailed and understandable comments.”

Gemini’s Program:

```
//EECS 348 Assignment 4
//Program reads
//Inputs: Puzzle Files
//Outputs: Writes the possible solutions for each sudoku text file into solutionN.txt
//Collaborators: None
//Other Sources: Claude, Gemini
//Author: Fabian Gonzalez
//Creation Date: October 4, 2026
//Revision Date: October 15, 2026
/* ------------------------------------
Revised by: Ana Gonzalez Yuil & AI Collaborator
Revision Date: 10.24.26
Sources used:
Revisions: Modified PuzzleReport to write solutions to solutionN.txt files, added digit
extraction for N, updated output formats for no-solutions and solution counts.
*/

#include <algorithm>
#include <array>
#include <cctype>
#include <cstddef>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
```

```cpp
// initialize name space for related sudoku dimensions
namespace {
constexpr int kBox = 3;
constexpr int kSize = kBox * kBox;                 // 9
constexpr int kCells = kSize * kSize;              // 81
constexpr unsigned kAllDigits = (1u << (kSize + 1)) - 2u;  // bits 1..9 set
}  // namespace

// ---------------------------------------------------------------------------
// Board: the grid plus bitmasks of digits already used per row/column/box.
// The masks make "which digits fit here?" an O(1) operation.
// ---------------------------------------------------------------------------
class Board { //class for sudoku board
public:
   Board() : cells_{}, rowUsed_{}, colUsed_{}, boxUsed_{}, consistent_(true) {}
//constructor method made of private ?s

   // Reads and validates a puzzle file. Returns false and fills `error` on
   // any problem (missing file, bad token, wrong cell count).
   bool load(const std::string& filename, std::string& error) {
       std::ifstream in(filename); //file given to program
       if (!in) { //checks if file can be opened, if not, return false and exit load()
           error = "cannot open file";
           return false;
       }
       std::string tok; //vague variable name (used to hold cells from file line)
       int n = 0; //used to count number of cells/current cell
       while (in >> tok) { //while tok stores a line from the file stream (in)
           if (n >= kCells) { //if number of cells (n) exceeds kCells (81), return false
               error = "more than " + std::to_string(kCells) + " cells";
               return false;
           }
           int d = parseCell(tok); // d is set to either -1 (invalid), 0 (empty), or 1-9
(valid number)
           if (d < 0) { //check if cell has an invalid char, return false
               error = "invalid token '" + tok + "' at cell " + std::to_string(n + 1);
               return false;
           }
           if (d > 0) addClue(n / kSize, n % kSize, d);
```

```
            ++n; //move to next cell
        }
        if (n < kCells) {
            error = "only " + std::to_string(n) + " of " + std::to_string(kCells) + " cells
found";
            return false;
        }
        return true;
    }

    bool isConsistent() const { return consistent_; }  // givens obey the rules?
    int at(int r, int c) const { return cells_[r * kSize + c]; }

    int blankCount() const {
        return static_cast<int>(std::count(cells_.begin(), cells_.end(), 0));
    }

    // Bitmask of digits that can legally go in (r,c).
    unsigned candidates(int r, int c) const {
        return kAllDigits & ~(rowUsed_[r] | colUsed_[c] | boxUsed_[boxOf(r, c)]);
    }

    void place(int r, int c, int d) { //places candidate value in cell using bitwise
operations
        cells_[r * kSize + c] = d;
        const unsigned bit = 1u << d;
        rowUsed_[r] |= bit;
        colUsed_[c] |= bit;
        boxUsed_[boxOf(r, c)] |= bit;
    }

    void remove(int r, int c, int d) { //removes candidate value from cell using bitwise
operations
        cells_[r * kSize + c] = 0;
        const unsigned bit = ~(1u << d);
        rowUsed_[r] &= bit;
        colUsed_[c] &= bit;
        boxUsed_[boxOf(r, c)] &= bit;
    }
```

```
    /*
    * @brief prints sudoku grid with readable ascii formatting
    * @param os output stream
    */
    void print(std::ostream& os) const {
        for (int r = 0; r < kSize; ++r) {
            if (r > 0 && r % kBox == 0) os << "------+-------+------\n";
            for (int c = 0; c < kSize; ++c) {
                if (c > 0 && c % kBox == 0) os << "| ";
                if (at(r, c) == 0) os << "_ ";
                else os << at(r, c) << ' ';
            }
            os << '\n';
        }
    }

    bool operator<(const Board& o) const { return cells_ < o.cells_; }  // for sorting, overrides < operator

    static int countBits(unsigned m) {
        int n = 0;
        for (; m; m &= m - 1) ++n;
        return n;
    }

private:
    static int boxOf(int r, int c) { return (r / kBox) * kBox + c / kBox; }

    // 0 = blank, 1-9 = digit, -1 = invalid token
    static int parseCell(const std::string& tok) {
        if (tok.size() != 1) return -1; //if the cell is not a single char/ empty char, return invalid
        const char ch = tok[0]; //convert tok from str to char
        if (ch == '_' || ch == '.' || ch == '0') return 0; //return 0 is char is character that indicates empty
        if (ch >= '1' && ch <= '9') return ch - '0';// if char is in range, return that char digit
        return -1; //otherwise return invalid
```

```
    }

    void addClue(int r, int c, int d) {
        if (candidates(r, c) & (1u << d)) {
            place(r, c, d);
        } else {
            consistent_ = false;        // duplicate in row/column/box
            cells_[r * kSize + c] = d; // keep it so the puzzle still prints
        }
    }

    std::array<int, kCells> cells_;
    unsigned rowUsed_[kSize], colUsed_[kSize], boxUsed_[kSize];
    bool consistent_;
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive DFS with backtracking. Holds results; never prints.
// At each node it branches on the blank cell with the fewest candidates
// (fail-first), which shrinks the search tree dramatically.
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    explicit SudokuSolver(const Board& start, std::size_t limit = 0)
        : work_(start), limit_(limit), stop_(false), truncated_(false) {}

    void run() {
        solutions_.clear();
        stop_ = truncated_ = false;
        if (!work_.isConsistent()) return;        // conflicting clues: no solution
(isConsistent = false)
        search(work_.blankCount());
        std::sort(solutions_.begin(), solutions_.end());  // deterministic order
    }

    const std::vector<Board>& solutions() const { return solutions_; }
    bool truncated() const { return truncated_; }  // more solutions existed than the limit

private:
```

```
void search(int blanksLeft) { //search for next blank cell
    if (blanksLeft == 0) {
        if (limit_ != 0 && solutions_.size() >= limit_) {
            truncated_ = stop_ = true;
            return;
        }
        solutions_.push_back(work_); //add to number of solutions
        return;
    }

    // Choose the most constrained blank cell.
    int bestR = -1, bestC = -1, bestCount = kSize + 1;
    unsigned bestMask = 0;
    for (int r = 0; r < kSize && bestCount > 1; ++r) {
        for (int c = 0; c < kSize; ++c) {
            if (work_.at(r, c) != 0) continue;
            const unsigned m = work_.candidates(r, c);
            const int n = Board::countBits(m);
            if (n < bestCount) {
                bestCount = n; bestR = r; bestC = c; bestMask = m;
                if (n <= 1) break;
            }
        }
    }
    if (bestCount == 0) return;               // dead end: backtrack immediately

    for (int d = 1; d <= kSize; ++d) {
        if (!(bestMask & (1u << d))) continue;
        work_.place(bestR, bestC, d);         // choose
        search(blanksLeft - 1);               // explore
        work_.remove(bestR, bestC, d);        // un-choose (backtrack)
        if (stop_) return;
    }
}

Board work_;
std::vector<Board> solutions_;
std::size_t limit_;
bool stop_, truncated_;
```

```
};

// ---------------------------------------------------------------------------
// PuzzleReport: loads one file, solves it, and writes the output into solutionN.txt
// ---------------------------------------------------------------------------
class PuzzleReport {
public:
   PuzzleReport(const std::string& filename, std::size_t limit)
       : filename_(filename), limit_(limit) {}

   // MODIFICATION: Executes puzzle solving and writes results to solutionN.txt
   // Returns false if the file could not be opened or processed.
   bool execute() const {
       // --- ADDITION: Extract puzzle number N from the filename (e.g., "puzzle1.txt" ->
"1") ---
       std::string nStr = "";
       for (char ch : filename_) {
           if (std::isdigit(static_cast<unsigned char>(ch))) {
               nStr += ch;
           }
       }
       if (nStr.empty()) {
           nStr = "1"; // Fallback default if filename contains no digits
       }

       // --- ADDITION: Construct output filename solutionN.txt and open file stream ---
       std::string outFilename = "solution" + nStr + ".txt";
       std::ofstream outFile(outFilename);
       if (!outFile) {
           std::cerr << "Error: Cannot open output file " << outFilename << "\n";
           return false;
       }

       Board puzzle;
       std::string error;
       if (!puzzle.load(filename_, error)) { // If file cannot be loaded, write error to
the solution file
           outFile << "Error: " << error << "\n";
           return false;
```

```
        }

        SudokuSolver solver(puzzle, limit_);
        solver.run();
        const std::vector<Board>& sols = solver.solutions();

        // --- MODIFICATION: Write "No solution found." if no solutions exist ---
        if (sols.empty()) {
            outFile << "No solution found.\n";
            return true;
        }

        // --- MODIFICATION: Write solved puzzle(s) followed by "Solutions found: n" ---
        for (std::size_t i = 0; i < sols.size(); ++i) {
            outFile << "Solution " << (i + 1) << ":\n";
            sols[i].print(outFile);
            if (i + 1 < sols.size()) {
                outFile << "\n";
            }
        }
        outFile << "Solutions found: " << sols.size() << "\n";

        return true;
    }

private:
    std::string filename_;
    std::size_t limit_;
};

int main(int argc, char** argv) {
    std::size_t limit = 0;
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) { //attempt to take input from command line
        const std::string arg = argv[i];
        if (arg == "--limit" && i + 1 < argc) limit = std::strtoul(argv[++i], nullptr, 10);
//strtoul converts string to unsigned long int
        else files.push_back(arg);
    }
```

```
    if (files.empty()) //otherwise, push file names (1-5) to vector manually
        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" + std::to_string(i) +
".txt");

    bool allOk = true; //indicates all file names ok
    for (const std::string& f : files) { //for each file in files
        PuzzleReport report(f, limit); //solves puzzle in file
        // MODIFICATION: Call execute() without std::cout, as output is now written to
solutionN.txt internally
        allOk = report.execute() && allOk;
    }
    return allOk ? 0 : 1; //return 0 or 1 based on allOK status
}
```

Copilot's Program:

```
//EECS 348 Assignment 4
//Program reads
//Inputs: Puzzle Files
//Outputs: Displays the possible solutions for the sudoku text file
//Collaboraters: None
//Other Sources: Claude, Gemini
//Author: Fabian Gonzalez
//Creation Date: October 4, 2026
//Revision Date: October 15, 2026
/* ------------------------------------
Revised by: Ana Gonzalez Yuil
Revision Date: 10.24.26
Sources used:
Revisions: additional comments,
*/


#include <algorithm>
#include <array>
#include <cstddef>
#include <cstdlib>
#include <fstream>
#include <iostream>
```

```cpp
#include <string>
#include <vector>
#include <cctype>

// initialize name space for related sudoku dimensions
namespace {
constexpr int kBox = 3;
constexpr int kSize = kBox * kBox;                  // 9
constexpr int kCells = kSize * kSize;               // 81
constexpr unsigned kAllDigits = (1u << (kSize + 1)) - 2u;  // bits 1..9 set
}  // namespace

// ---------------------------------------------------------------------------
// Board: the grid plus bitmasks of digits already used per row/column/box.
// The masks make "which digits fit here?" an O(1) operation.
// ---------------------------------------------------------------------------
class Board { //class for sudoku board
public:
   Board() : cells_{}, rowUsed_{}, colUsed_{}, boxUsed_{}, consistent_(true) {}
//constructor method made of private ?s

   // Reads and validates a puzzle file. Returns false and fills `error` on
   // any problem (missing file, bad token, wrong cell count).
   bool load(const std::string& filename, std::string& error) {
       std::ifstream in(filename); //file given to program
       if (!in) { //checks if file can be opened, if not, return false and exit load()
           error = "cannot open file";
           return false;
       }
       std::string tok; //vauge variable name (used to hold cells from file line)
       int n = 0; //used to count number of cells/current cell
       while (in >> tok) { //while tok stores a line from the file stream (in)
           if (n >= kCells) { //if number of cells (n) exceeds kCells (81), return false
               error = "more than " + std::to_string(kCells) + " cells";
               return false;
           }
           int d = parseCell(tok); // d is set to either -1 (invalid), 0 (empty), or 1-9
(valid number)
           if (d < 0) { //check if cell has an invalid char, return false
```

```
                error = "invalid token '" + tok + "' at cell " + std::to_string(n + 1);
                return false;
            }
            if (d > 0) addClue(n / kSize, n % kSize, d);
            ++n; //move to next cell
        }
        if (n < kCells) {
            error = "only " + std::to_string(n) + " of " + std::to_string(kCells) + " cells
found";
            return false;
        }
        return true;
    }

    bool isConsistent() const { return consistent_; }  // givens obey the rules?
    int at(int r, int c) const { return cells_[r * kSize + c]; }

    int blankCount() const {
        return static_cast<int>(std::count(cells_.begin(), cells_.end(), 0));
    }

    // Bitmask of digits that can legally go in (r,c).
    unsigned candidates(int r, int c) const {
        return kAllDigits & ~(rowUsed_[r] | colUsed_[c] | boxUsed_[boxOf(r, c)]);
    }

    void place(int r, int c, int d) { //places candidate value in cell using bitwise
operations
        cells_[r * kSize + c] = d;
        const unsigned bit = 1u << d;
        rowUsed_[r] |= bit;
        colUsed_[c] |= bit;
        boxUsed_[boxOf(r, c)] |= bit;
    }

    void remove(int r, int c, int d) { //removes candidate value from cell using bitwise
operations
        cells_[r * kSize + c] = 0;
        const unsigned bit = ~(1u << d);
```

```
        rowUsed_[r] &= bit;
        colUsed_[c] &= bit;
        boxUsed_[boxOf(r, c)] &= bit;
    }

    /*
    * @brief prints sudoku grid with readable ascii formatting
    * @param os output stream
    */
    void print(std::ostream& os) const {
        for (int r = 0; r < kSize; ++r) {
            if (r > 0 && r % kBox == 0) os << "------+-------+------\n";
            for (int c = 0; c < kSize; ++c) {
                if (c > 0 && c % kBox == 0) os << "| ";
                if (at(r, c) == 0) os << "_ ";
                else os << at(r, c) << ' ';
            }
            os << '\n';
        }
    }

    bool operator<(const Board& o) const { return cells_ < o.cells_; }  // for sorting,
overrides < operator

    static int countBits(unsigned m) {
        int n = 0;
        for (; m; m &= m - 1) ++n;
        return n;
    }

private:
    static int boxOf(int r, int c) { return (r / kBox) * kBox + c / kBox; }

    // 0 = blank, 1-9 = digit, -1 = invalid token
    static int parseCell(const std::string& tok) {
        if (tok.size() != 1) return -1; //if the cell is not a single char/ empty char,
return invalid
        const char ch = tok[0]; //convert tok from str to char
        if (ch == '_' || ch == '.' || ch == '0') return 0; //return 0 is char is character
```

```
that indicates empty
        if (ch >= '1' && ch <= '9') return ch - '0';// if char is in range, return that char
digit
        return -1; //otherwise return invalid
    }

    void addClue(int r, int c, int d) {
        if (candidates(r, c) & (1u << d)) {
            place(r, c, d);
        } else {
            consistent_ = false;       // duplicate in row/column/box
            cells_[r * kSize + c] = d; // keep it so the puzzle still prints
        }
    }

    std::array<int, kCells> cells_;
    unsigned rowUsed_[kSize], colUsed_[kSize], boxUsed_[kSize];
    bool consistent_;
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive DFS with backtracking. Holds results; never prints.
// At each node it branches on the blank cell with the fewest candidates
// (fail-first), which shrinks the search tree dramatically.
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    explicit SudokuSolver(const Board& start, std::size_t limit = 0)
        : work_(start), limit_(limit), stop_(false), truncated_(false) {}

    void run() {
        solutions_.clear();
        stop_ = truncated_ = false;
        if (!work_.isConsistent()) return;        // conflicting clues: no solution
(isConsistent = false)
        search(work_.blankCount());
        std::sort(solutions_.begin(), solutions_.end());  // deterministic order
    }
```

```
    const std::vector<Board>& solutions() const { return solutions_; }
    bool truncated() const { return truncated_; }  // more solutions existed than the limit

private:
    void search(int blanksLeft) { //search for next blank cell
        if (blanksLeft == 0) {
            if (limit_ != 0 && solutions_.size() >= limit_) {
                truncated_ = stop_ = true;
                return;
            }
            solutions_.push_back(work_); //add to number of solutions
            return;
        }

        // Choose the most constrained blank cell.
        int bestR = -1, bestC = -1, bestCount = kSize + 1;
        unsigned bestMask = 0;
        for (int r = 0; r < kSize && bestCount > 1; ++r) {
            for (int c = 0; c < kSize; ++c) {
                if (work_.at(r, c) != 0) continue;
                const unsigned m = work_.candidates(r, c);
                const int n = Board::countBits(m);
                if (n < bestCount) {
                    bestCount = n; bestR = r; bestC = c; bestMask = m;
                    if (n <= 1) break;
                }
            }
        }
        if (bestCount == 0) return;                // dead end: backtrack immediately

        for (int d = 1; d <= kSize; ++d) {
            if (!(bestMask & (1u << d))) continue;
            work_.place(bestR, bestC, d);          // choose
            search(blanksLeft - 1);                // explore
            work_.remove(bestR, bestC, d);         // un-choose (backtrack)
            if (stop_) return;
        }
    }
```

```
    Board work_;
    std::vector<Board> solutions_;
    std::size_t limit_;
    bool stop_, truncated_;
};


// ---------------------------------------------------------------------------
// ADDED FUNCTION
// Extracts the puzzle number from a filename such as "puzzle3.txt".
// This number is used to create the output file "solution3.txt".
// If no number is found, returns 0.
// ---------------------------------------------------------------------------
int extractPuzzleNumber(const std::string& filename) {
    std::string digits;

    // Collect every digit appearing in the filename.
    for (char ch : filename) {
        if (std::isdigit(ch)) {
            digits += ch;
        }
    }

    return digits.empty() ? 0 : std::stoi(digits);
}

// ---------------------------------------------------------------------------
// PuzzleReport: loads one file, solves it, prints the required output.
// ---------------------------------------------------------------------------
class PuzzleReport {
public:
    PuzzleReport(const std::string& filename, std::size_t limit)
        : filename_(filename), limit_(limit) {}

    // Returns false if the file could not be used.
    bool execute(std::ostream& os) const {

    os << "================================================\n";
    os << "Puzzle file: " << filename_ << "\n";
```

```
Board puzzle;
std::string error;

if (!puzzle.load(filename_, error)) {
    os << "Error: " << error << "\n\n";
    return false;
}

os << "Puzzle:\n";
puzzle.print(os);

SudokuSolver solver(puzzle, limit_);
solver.run();

const std::vector<Board>& sols = solver.solutions();

// ------------------------------------------------------------------
// ADDED CODE
// Create output file named solutionN.txt where N matches the puzzle
// file number (puzzle1.txt -> solution1.txt).
// ------------------------------------------------------------------
int puzzleNumber = extractPuzzleNumber(filename_);

std::string solutionFileName =
    "solution" + std::to_string(puzzleNumber) + ".txt";

std::ofstream solutionFile(solutionFileName);

if (!solutionFile) {
    os << "Unable to create " << solutionFileName << "\n";
    return false;
}
// ------------------------------------------------------------------

if (sols.empty()) {

    // --------------------------------------------------------------
    // MODIFICATION:
```

```
    // Write required message to the solution file when no solutions
    // are found.
    // ------------------------------------------------------------------
    solutionFile << "No solution found.\n";
    // ------------------------------------------------------------------

    os << "\nNo solution found\n\n";
    return true;
}

os << "\n" << sols.size() << " solution(s) found"
   << (solver.truncated() ? " (stopped at limit; more exist)" : "")
   << "\n";

// ------------------------------------------------------------------
// MODIFICATION:
// Write every solution to the solution file.
// ------------------------------------------------------------------
for (std::size_t i = 0; i < sols.size(); ++i) {

    os << "\nSolution " << (i + 1) << ":\n";
    sols[i].print(os);

    solutionFile << "Solution " << (i + 1) << ":\n";
    sols[i].print(solutionFile);
    solutionFile << "\n";
}

// ------------------------------------------------------------------
// ADDED CODE
// Write the total number of solutions after all solutions have been
// written, exactly as required in the assignment specification.
// ------------------------------------------------------------------
solutionFile << "Solutions found: "
             << sols.size()
             << "\n";
// ------------------------------------------------------------------

os << "\n";
```

```
    return true;
}

private:
    std::string filename_;
    std::size_t limit_;
};

int main(int argc, char** argv) {
    std::size_t limit = 0;
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) { //attempt to take input from command line
        const std::string arg = argv[i];
        if (arg == "--limit" && i + 1 < argc) limit = std::strtoul(argv[++i], nullptr, 10);
//strtoul converts string to unsigned long int
        else files.push_back(arg);
    }
    if (files.empty()) //otherwise, push file names (1-5) to vector manually
        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" + std::to_string(i) + ".txt");

    bool allOk = true; //indicates all file names ok
    for (const std::string& f : files) { //for each file in files
        PuzzleReport report(f, limit); //solves puzzle in file
        allOk = report.execute(std::cout) && allOk;
    }
    return allOk ? 0 : 1; //return 0 or 1 based on allOK status
}
```

Analysis:

1. Gemini
    a. Correctness:
        i. Gemini modified the execute() method in the PuzzleReport class to make the printing method output into a text file instead.
        ii. When given expected input: The modified program still properly solves the sudoku puzzle as expected by using a DFS algorithm with backtracking. The output files are properly named and formatted correctly, containing either the solutions and the number of solutions found, or the text “No solution found.”
        iii. When given unexpected input (file does not exist, dimensions are not 9x9, sudoku grid with no empty spaces): When the puzzle file does not exist or the puzzle files are not 9x9, the program writes the relevant error to the file instead of a solution. When the puzzle has no empty spaces (already fully solved), the solved puzzle is listed as a solution and still written to the solution file (this is technically correct).
    b. Execution Time:
        i. O(n) → The modifications added iterate through the puzzle file name to find a digit char, so execution time depends on the length of filename to search for. The program also iterates through the number of solutions for each puzzle, so the execution time for each file also varies there.
    c. Space Complexity:
        i. O(1) → The only two variables created are a string to hold the full filename and a char to hold the number of the file to name the solution file; they basically are the same size every time.
    d. Maintainability:
        i. output file namer only searches for first digit char in file name to name solution file – what do we do when we get to double digits?
        ii. Comments explain what each addition/ modification does
2. CoPilot
    a. Correctness:
        i. Copilot also modifies the execute() method from the PuzzleReport class, and also adds a separate method for obtaining the filename number. However, it prints to both the terminal and to the solution files, which is not the fully desired result.
        ii. When given expected input: The modified program still properly solves the sudoku puzzle as expected by using a DFS algorithm with backtracking. The output files are properly named and formatted correctly, containing either the solutions and the number of solutions found, or the text “No solution found.”
        iii. When given unexpected input (file does not exist, dimensions are not 9x9, sudoku grid with no empty spaces): When the file does not exist or dimensions are not 9x9, the program informs the user of the error through the terminal, but no solution file is created. When the sudoku puzzle is

already solved (no empty cells), the program lists the solved puzzle as a solution and is still written to the solution file (this is technically correct).

b. Execution Time:
    i. O(n) → The modifications added iterate through the puzzle file name to find digit char, so execution time depends on the length of filename to search for. The program also iterates through the number of solutions for each puzzle, so the execution time for each file also varies there.
c. Space Complexity:
    i. O(1) → The only two variables created are a string to hold the full filename and a char to hold the number of the file to name the solution file; they basically are the same size every time.
d. Maintainability:
    i. A separate method to collect all the file numbers makes better readability.
    ii. Comments indicate additions/modifications and what they do

Program Selected: Gemini

1. Justification
    a. The method used to obtain the file number is more efficient in Gemini's program.
    b. Overall, Gemini's modifications are more correct / efficient and do not keep unnecessary program functionalities (ex: printing to terminal).
2. Changes Made
    a. Added additional comments, combined Gemini modifications with original program